Laboratório Prolog · Aula 13

Bruno Vinicius Coutinho Miante da Silva · RA 24151605-2 · Turma ESOFT C
Gerado em 20/10/2026, 19:52:54 · tempo registrado no cronômetro: 0:00

Preparação: Programar descrevendo, não instruindo

Previsão: O que paradigma(L, funcional) e paradigma(java, logico) vão responder?
paradigma(L, funcional) consulta quais linguagens L foram registradas como funcionais; paradigma(java, logico) pergunta se existe um fato dizendo que Java é lógico. A segunda consulta tende a falhar (false) se esse fato não estiver na base.

Saída do ola.pl
A saída esperada de ola.pl é a mensagem de saudação definida no arquivo, normalmente algo como "Olá, mundo!". O texto exato depende do comando write/format usado no código.

0.1 Qual é a diferença entre paradigma(prolog, P) e paradigma(prolog, logico)? O que cada uma pergunta?
paradigma(prolog, P) procura todos os paradigmas associados a Prolog e pode retornar o valor de P. paradigma(prolog, logico) pergunta se Prolog está registrado especificamente como lógico; é uma consulta de verificação, que retorna true ou false.

0.2 paradigma(java, logico) deu false: Java não é lógico ou o programa não sabe isso? Justifique.
O resultado false significa que a base de conhecimento não contém um fato ou regra que prove paradigma(java, logico). Isso não demonstra, por si só, que Java jamais possa ser usado em lógica; significa apenas que a consulta não pode ser provada a partir do programa carregado.

Etapa 1: Uma base de fatos e perguntas sobre ela

Previsão: Quantas respostas terá mulher(Quem)? E quais pares terá (mae(M, F), mulher(F))?
A quantidade de respostas é igual ao número de fatos mulher/1 existentes na base familia.pl. Os pares são as combinações em que mae(M, F) e mulher(F) são verdadeiros ao mesmo tempo. A lista exata depende dos fatos definidos no arquivo.

Saída do familia.pl

A saída depende dos fatos cadastrados em familia.pl. A execução lista os fatos de mulher/1 e as combinações que satisfazem a consulta composta; confira os nomes exibidos pelo seu arquivo.

1.1 Escreva em português o que significa a consulta (mae(Mae, Filha), mulher(Filha)) e quantas respostas ela teve.

A consulta significa: encontrar uma pessoa Mae que seja mãe de Filha e verificar se Filha é mulher. Cada resposta apresenta uma combinação que satisfaz as duas condições. A quantidade exata depende dos fatos de familia.pl e deve ser conferida na execução.

1.2 pai(maria, jake) deu false mas a base não menciona maria. Que pressuposto explica? Por que pode ser um problema?

O Prolog usa a hipótese do mundo fechado: se um fato não pode ser provado pela base, a consulta falha. Isso pode ser problemático porque ausência de informação é tratada como falsidade, mesmo quando o dado apenas não foi cadastrado.

1.3 Em que ordem as respostas de mulher(Quem) apareceram? O que determina essa ordem?

As respostas aparecem na ordem em que o Prolog encontra os fatos e percorre as cláusulas da base, normalmente de cima para baixo e da esquerda para a direita. Portanto, a ordem dos fatos no arquivo influencia a ordem dos resultados.

Etapa 2: Regras: ensinando o Prolog a deduzir

Previsão: Quais respostas terão tio(X, jake) e primo(leo, X)?

tio(X, jake) procura pessoas X que satisfaçam a regra de tio em relação a Jake. primo(leo, X) procura pessoas X que sejam primas ou primos de Leo conforme a regra definida. Os nomes concretos dependem dos fatos e regras presentes em regras.pl.

Previsão: Quais unificações vão falhar: f(X,X)=f(a,b), f(X,X)=f(a,Y), [A|B]=[], 2+3=5?

f(X,X)=f(a,b) falha porque o mesmo X teria de ser simultaneamente a e b. f(X,X)=f(a,Y) unifica: X=a e Y=a. [A|B]=[] falha porque uma lista vazia não tem cabeça e cauda. 2+3=5 unifica sintaticamente os termos e falha, pois 2+3 é uma estrutura, não o número 5.

Saídas de regras.pl e unificacao.pl

As saídas exatas dependem das cláusulas de regras.pl e unificacao.pl. Os resultados conceituais das consultas de unificação estão respondidos abaixo; para os nomes de tio e primo, é necessário usar os fatos concretos desses arquivos.

2.1 Por que ancestral precisa de duas cláusulas? Qual é o caso base e qual é o caso recursivo?

São necessárias duas cláusulas para cobrir o caso direto e os ancestrais mais distantes. Caso base: ancestral(X,Y) :- pai(X,Y) (e/ou mãe, conforme o programa). Caso recursivo: ancestral(X,Y) :- pai(X,Z), ancestral(Z,Y), repetindo a busca pela geração anterior.

2.2 Apague X \== Y da regra irmao e consulte irmao(jake, X). O que mudou? Por que a meta é necessária?

Sem X \== Y, a regra pode considerar uma pessoa irmã de si mesma, dependendo dos fatos que satisfazem os demais objetivos. A meta X \== Y impede que os dois argumentos sejam idênticos e exclui esse resultado. A forma mais robusta é também modelar a relação de parentesco de modo a evitar essa situação.

2.3 Por que f(X,X)=f(a,b) não unifica mas f(X,X)=f(a,Y) unifica? Quais valores ficam em X e Y?

Na primeira expressão, o mesmo X precisa corresponder aos dois argumentos, mas a e b são diferentes, então não há unificação. Na segunda, X pode ser a e Y também pode ser a, pois não existe conflito: o resultado é X=a, Y=a.

2.4 avo responde em várias direções. Isso é possível numa função de Java ou Scheme? Explique a diferença.

Em Prolog, avo pode ser consultado em diferentes direções: pode encontrar o avô a partir do neto ou procurar netos a partir do avô, conforme os fatos e a regra. Em funções comuns de Java ou Scheme, a execução costuma seguir argumentos de entrada e produzir um retorno; não há automaticamente a mesma busca relacional reversível.

Etapa 3: Listas: o mesmo padrão do Scheme, em formato de relação

Previsão: O que concatena(X, Y, [1,2,3]) vai responder? Quantas respostas?

concatena(X,Y,[1,2,3]) divide a lista final em prefixo X e sufixo Y. As quatro respostas são X=[]/Y=[1,2,3]; X=[1]/Y=[2,3]; X=[1,2]/Y=[3]; e X=[1,2,3]/Y=[].

Saída do listas.pl

Caso base: tamanho([],0). Passo recursivo: tamanho([_|Cauda],N) :- tamanho(Cauda,N1), N is N1+1. A cada chamada fica pendente contar o elemento retirado da cabeça e somá-lo ao tamanho da cauda quando a recursão retornar.

3.1 Em tamanho, qual é o caso base e qual é o passo recursivo? O que fica pendente a cada chamada?

A saída de listas.pl deve mostrar as soluções geradas pelas regras de listas. Para concatena(X,Y,[1,2,3]), as quatro divisões possíveis da lista estão descritas na resposta seguinte.

3.2 Por que concatena(X,Y,[1,2,3]) tem 4 respostas? Compare com anexar do Scheme: o que ele não consegue fazer?

Há quatro respostas porque uma lista com três elementos pode ser separada em quatro pontos: antes do primeiro elemento, entre os elementos e depois do último. A relação concatena pode descobrir os dois segmentos a partir da lista final; uma função tradicional

anexar no Scheme geralmente recebe os argumentos e retorna o resultado, não resolve automaticamente a operação inversa.

3.3 membro(X,[a,b,c]) gerou a, b e c sem laço. Que mecanismo do Prolog fez isso?

A regra membro é definida recursivamente: o elemento pode ser a cabeça da lista ou estar na cauda. O Prolog tenta as cláusulas e usa retrocesso (backtracking) para encontrar a, b e c, sem precisar de um laço explícito.

Etapa 4: Aritmética: onde o Prolog deixa de ser relacional

Previsão: O que mostram X = 2 + 3 e X is 2 + 3? E fatorial(N, 120)?

X = 2+3 associa X ao termo simbólico +(2,3), sem calcular. X is 2+3 avalia a expressão aritmética e associa X a 5. fatorial(N,120) tenta encontrar N, mas uma regra aritmética com is normalmente exige que as variáveis do lado direito estejam instanciadas; por isso pode ocorrer erro de variável não instanciada.

Saída do aritmetica.pl

= verifica unificação: X = 2+3 faz X representar a expressão 2+3. is avalia o lado direito: X is 2+3 resulta em X=5. =:= compara valores numéricos após avaliação: 2+3 =:= 5 é verdadeiro, mesmo que as expressões tenham formas diferentes.

4.1 Explique com exemplos a diferença entre =, is e =:=.

fatorial(N,120) pode dar erro porque a regra usa avaliação aritmética (is) com N ainda desconhecido; Prolog não resolve automaticamente a equação inversa. concatena(X,[3],[1,2,3]) funciona porque a concatenação é definida por relações estruturais e unificação, sem precisar calcular uma expressão numérica com variáveis desconhecidas.

4.2 Por que fatorial(N,120) dá erro e concatena(X,[3],[1,2,3]) funcionou? O que há de diferente na aritmética?

A saída de aritmetica.pl deve mostrar a diferença entre unificação e avaliação aritmética; expressões como is exigem que os valores necessários já estejam instanciados.

4.3 Compare o fatorial em Prolog com o fact em Haskell: o que é igual e o que é diferente?

Ambos definem o fatorial recursivamente, com caso base e caso recursivo. Em Prolog, a relação costuma usar fatos/regras e operações aritméticas explícitas, como is; a direção de consulta é limitada quando a aritmética exige argumentos instanciados. Em Haskell, fact é uma função que recebe um valor e retorna um resultado, com tipagem estática e avaliação funcional.

Etapa 5: Controlando (e sofrendo com) a busca

Previsão: qual de solteiro1/solteiro2 dá false? maximo2(5,3,M) uma ou duas respostas? maximo3(5,3,3) true ou false?

solteiro1 pode produzir uma conclusão incorreta por causa da negação por falha e da ordem dos objetivos; solteiro2, com a ordem apropriada dos testes, pode falhar quando a pessoa não satisfaz as condições. maximo2(5,3,M) normalmente encontra M=5, mas pode oferecer respostas repetidas se as cláusulas não forem mutuamente exclusivas. maximo3(5,3,3) retorna true se a cláusula com corte aceita o terceiro argumento sem verificar corretamente que ele é o máximo.

Saída de controle.pl e mensagem de ancestral_ruim

A saída exata depende das cláusulas de controle.pl. Em geral, o rastro mostra tentativas de satisfazer objetivos, falhas e retrocesso para alternativas anteriores. ancestral_ruim pode entrar em recursão sem progresso e continuar até estourar a pilha; ancestral_bom coloca a condição que reduz o problema antes da chamada recursiva.

Previsão: Quantas colorações existem com 3 cores? E com 2 cores?

O número exato de colorações depende das regiões e das adjacências descritas em mapa.pl. O Prolog gera atribuições de cores e rejeita as que dão a mesma cor a regiões vizinhas. Com três cores geralmente há soluções; com duas, pode haver menos ou nenhuma, conforme o grafo. É necessário executar o arquivo para informar a contagem exata.

Saída do mapa.pl

O rastro mostra o Prolog tentando combinações de X e Y que satisfazem mesmo_gosto/2. Quando uma meta falha, ele volta (backtracking) à meta anterior e tenta a próxima cláusula ou fato disponível. A meta que falha em cada passo depende da ordem concreta das cláusulas em controle.pl.

5.1 Com o rastro do bloco A, explique como o Prolog encontrou as respostas de mesmo_gosto(X,Y). Que meta falhou e para onde voltou?

solteiro2 tende a dar false quando verifica primeiro que a pessoa não é casada e depois exige uma condição que não pode ser provada; a ordem e o uso de negação por falha alteram o resultado. solteiro1 pode afirmar que Osvaldo é solteiro porque a ausência de prova de casamento é interpretada como prova de que não é casado, mesmo sem informação completa sobre ele.

5.2 Por que solteiro2 dá false e solteiro1 não? Por que solteiro1 afirma que osvaldo é solteiro?

Com três cores, o programa procura todas as atribuições válidas para as regiões do mapa. A quantidade depende das regiões e das relações de vizinhança definidas em mapa.pl.

5.3 maximo3(5,3,3) deu true. Explique por que o corte não protegeu esse uso. Como você corrigiria?

O corte (!) só impede o retrocesso depois de ser alcançado; ele não corrige uma condição lógica mal formulada. Se maximo3(5,3,3) deu true, a cláusula aceitou M=3 sem garantir que M fosse o maior. Corrigiria verificando M=5 quando A>=B e M=B apenas quando B>A, usando condições consistentes e colocando o corte após a comparação correta.

5.4 Por que ancestral_ruim estoura a pilha e ancestral_bom não, se dizem a mesma coisa logicamente?

ancestral_ruim coloca a chamada recursiva antes de uma condição que limite a busca ou usa uma direção de recursão que não reduz o problema, causando chamadas infinitas. ancestral_bom usa um caso base e uma recursão que avança para um parente anterior, permitindo que a busca termine. As regras podem ser logicamente equivalentes, mas a ordem das metas afeta a execução do Prolog.

5.5 Quantas colorações com 3, 2 e 4 cores? Por que o programa é declarativo apesar de fazer busca?

A quantidade exata para 3, 2 e 4 cores depende do mapa e das adjacências em mapa.pl; deve ser obtida executando a consulta de todas as soluções. O programa é declarativo porque descreve as restrições que uma coloração precisa cumprir; o mecanismo de busca do Prolog procura as atribuições que satisfazem essas relações.

Fechamento: O que você viu e onde isso se encaixa

6.1 Explique a diferença entre o paradigma lógico e o funcional usando concatena e anexar.

No paradigma lógico, concatena descreve uma relação entre duas listas e a lista resultante, permitindo descobrir partes desconhecidas por unificação e retrocesso. No paradigma funcional, anexar recebe entradas e calcula uma saída. Assim, Prolog enfatiza relações e consultas, enquanto Scheme enfatiza a aplicação de funções.

6.2 Cite duas deficiências do Prolog que você observou, ligando cada uma a um programa.

Duas limitações observadas são: (1) a hipótese do mundo fechado/negação por falha, vista em solteiro1, que pode concluir algo apenas porque não há prova em contrário; (2) a sensibilidade à ordem das metas e à recursão, vista em ancestral_ruim, que pode causar busca infinita ou estouro de pilha.

6.3 Para que tipo de problema você escolheria Prolog em vez de Java? E para qual não escolheria?

Escolheria Prolog para problemas de regras, relações, dedução, planejamento ou busca de combinações, como parentesco e coloração de mapas. Não o escolheria como primeira opção para interfaces gráficas, aplicativos comerciais convencionais ou tarefas numéricas em que o fluxo de execução e o desempenho previsível sejam prioritários; Java pode ser mais adequado nesses casos.